

Project Proposal

Smart Offline Expense Manager with Secure Automatic Cloud Backup

1. Project Title

Smart Offline Expense Manager with Encrypted Automatic Cloud Backup for Android and iOS

2. Introduction

Personal finance management has become an essential part of modern life. Most existing expense tracking applications rely heavily on continuous internet connectivity and centralized cloud servers. These systems introduce several challenges such as privacy concerns, hosting costs, data security risks, and limited usability during internet outages.

This project proposes the development of a cross-platform offline-first mobile application that enables users to manage personal expenses securely without requiring permanent internet connectivity or dedicated server infrastructure.

The application will operate entirely offline during normal usage while automatically creating encrypted backups to Google Drive whenever internet connectivity becomes available. The system is designed to prioritize user privacy, low operational cost, data security, and lightweight performance.

The proposed application will support both Android and iOS devices using a single Flutter codebase.

3. Problem Statement

Most finance tracking applications:

- Depend on constant internet connectivity
- Store sensitive financial information on external servers
- Require subscription-based cloud services
- Introduce privacy concerns
- Become unusable during network failures
- Increase development and maintenance costs through backend infrastructure

Users require a lightweight and privacy-focused financial management solution that:

- Works completely offline
- Does not require dedicated hosting

- Automatically protects user data
- Minimizes internet dependency
- Supports multiple mobile platforms

Therefore, there is a need for a secure offline-first expense management system with automatic encrypted backup functionality.

4. Objectives

4.1 Main Objective

To develop a cross-platform offline-first expense management mobile application with secure automatic encrypted cloud backup functionality.

4.2 Specific Objectives

- Develop a single codebase application for Android and iOS
 - Store financial data locally on the device
 - Provide secure offline expense tracking
 - Implement automatic encrypted cloud backup
 - Generate expense reports and analytics
 - Minimize battery, storage, and network usage
 - Provide secure authentication mechanisms
 - Enable backup restoration functionality
-

5. Scope of the System

Included Features

- Expense tracking
- Expense categorization
- Budget management
- Monthly and yearly analytics
- Offline-first operation
- Local database storage
- Automatic encrypted cloud backup
- Google Drive integration
- Backup restoration
- PIN and biometric authentication
- Dark mode support

Excluded Features

- Multi-user collaboration
- Online banking integration

- Payment gateway integration
 - Real-time cloud synchronization
 - Cryptocurrency tracking
 - Web dashboard
-

6. Proposed Solution

The proposed solution is a Flutter-based cross-platform mobile application that stores all user data locally using an optimized offline database.

The application will continuously function without internet access while monitoring network availability. When internet connectivity becomes available through Wi-Fi or mobile data, the application automatically performs encrypted backups to Google Drive using secure APIs.

This architecture eliminates the need for dedicated servers while ensuring reliable data protection.

7. Functional Requirements

7.1 Expense Management

- Add expenses
- Edit expenses
- Delete expenses
- Search expenses
- Filter by category and date

7.2 Category Management

- Create categories
- Modify categories
- Delete categories

7.3 Analytics and Reporting

- Monthly summaries
- Spending trends
- Category-wise reports
- Charts and visual analytics

7.4 Backup System

- Detect internet connectivity
- Generate encrypted backup files
- Upload backups automatically
- Restore previous backups
- Maintain backup history

7.5 Security Features

- PIN lock
 - Fingerprint authentication
 - Face ID authentication (iOS)
 - Encrypted local storage
 - Encrypted cloud backups
-

8. Non-Functional Requirements

Performance

- Fast local database operations
- Low battery consumption
- Lightweight application size

Security

- AES-256 encryption
- Secure authentication
- Secure backup transfer

Reliability

- Offline-first availability
- Automatic backup retry mechanism
- Data recovery support

Portability

- Android support
 - iOS support
 - Single codebase maintainability
-

9. System Architecture

Architecture Type

Offline-First Cross-Platform Mobile Architecture

Architecture Layers

Presentation Layer

Flutter UI Components

State Management Layer

Riverpod

Business Logic Layer

Service and Repository Pattern

Local Storage Layer

Isar Database

Backup Layer

Encryption Service and Google Drive Service

Background Task Layer

WorkManager and Connectivity Monitoring

10. Technology Stack

Component	Technology
Framework	Flutter
Programming Language	Dart
Local Database	Isar Database
State Management	Riverpod
Charts	fl_chart
Connectivity Detection	connectivity_plus
Background Tasks	Workmanager
Encryption	encrypt package
Authentication	local_auth
Cloud Backup	Google Drive API
Dependency Management	pub.dev packages

11. Database Design

Expense Collection

Field	Type
id	Integer
title	String
amount	Double
category	String
date	DateTime
note	String
createdAt	Timestamp

Category Collection

Field	Type
id	Integer
name	String
color	String

Backup Metadata Collection

Field	Type
id	Integer
lastBackup	Timestamp
backupStatus	Boolean
pendingChanges	Boolean

12. Data Flow

Expense Recording Flow

User Input → Validation → Isar Database → UI Update

Automatic Backup Flow

Expense Modification → Mark Pending Backup → Internet Available → Encrypt Backup → Upload to Google Drive → Update Backup Status

Restore Flow

Google Drive → Download Backup → Decrypt File → Replace Local Database → Refresh Application

13. Backup System Design

Backup Trigger Conditions

- Internet connection available
 - Pending data changes exist
 - Device battery not critically low
-

Backup Procedure

1. Create local database copy
 2. Compress backup file
 3. Encrypt using AES-256
 4. Upload to Google Drive
 5. Store backup metadata
-

Backup Retention Policy

- Keep latest 7 backups
 - Automatically remove older backups
-

14. Security Design

Local Security

- Application PIN lock
- Fingerprint authentication
- Face ID authentication
- Secure local file access

Cloud Security

- AES-256 encrypted backups
- Secure Google authentication
- Protected API communication

Privacy Features

- No third-party servers
 - No analytics tracking
 - User-controlled backups
-

15. User Interface Design

Main Screens

- Splash Screen
 - Dashboard Screen
 - Add Expense Screen
 - Expense History Screen
 - Analytics Screen
 - Categories Screen
 - Backup Settings Screen
 - Restore Backup Screen
 - Authentication Screen
-

16. Application Workflow

Daily Usage

User opens app → Records expenses → Data saved locally → Reports generated offline

Backup Workflow

Internet becomes available → App detects connectivity → Backup generated → Backup encrypted → Uploaded to Google Drive automatically

17. Storage Analysis

Estimated Storage Usage

Data Type	Estimated Size
1,000 expenses	< 1 MB
10,000 expenses	5–10 MB
Encrypted backups	2–10 MB
Full application size	25–50 MB

18. Development Methodology

Agile Incremental Development Model

Development Phases

Phase 1

UI and local database implementation

Phase 2

Expense management features

Phase 3

Analytics and chart integration

Phase 4

Backup and encryption system

Phase 5

Testing and optimization

19. Testing Strategy

Unit Testing

- Database operations
- Encryption functions
- Backup logic

Integration Testing

- Google Drive integration
- Restore functionality

UI Testing

- Navigation flow
- Form validation
- Authentication process

Performance Testing

- Battery usage
 - Storage optimization
 - Large dataset handling
-

20. Advantages of the Proposed System

- Fully offline operation
 - Cross-platform support
 - No hosting cost
 - Enhanced privacy
 - Secure encrypted backups
 - Lightweight architecture
 - Fast performance
 - Automatic data protection
-

21. Limitations

- Limited background execution on iOS
 - Single-user architecture
 - No real-time synchronization
 - Dependent on Google Drive availability
-

22. Future Enhancements

- Multi-device synchronization
 - OCR receipt scanning
 - AI-based spending prediction
 - Voice expense entry
 - PDF and Excel export
 - Shared family accounts
 - Budget recommendation system
 - Multi-currency support
-

23. Estimated Development Timeline

Phase	Duration
Requirement Analysis	1 Week
UI/UX Design	1 Week
Database Development	1 Week
Expense Features	2 Weeks
Analytics Integration	1 Week
Backup System Development	2 Weeks
Security Features	1 Week
Testing and Optimization	1 Week

Total Estimated Duration: 10 Weeks

24. Expected Outcomes

The completed system will:

- Operate entirely offline
 - Support Android and iOS devices
 - Automatically backup encrypted data
 - Protect user privacy
 - Minimize storage and network usage
 - Provide efficient financial tracking
-

25. Conclusion

The Smart Offline Expense Manager is a privacy-focused, lightweight, and secure financial management application designed for modern mobile platforms. By combining offline-first architecture with encrypted cloud backup functionality, the system provides reliable personal finance management without requiring dedicated backend infrastructure.

The proposed solution offers an efficient, scalable, and cost-effective approach to secure mobile expense management for both Android and iOS users.